

Visualisateur 3D

Raphaël Jorel

Université de Bordeaux

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

Rappels

Traitements

Logiciel

Difficultés et limites

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

Rappels

Traitements

Logiciel

Difficultés et limites

Contexte

- ▶ *Acquisition* : Récupération de données à partir du monde réel (CT Scan, X-ray, ...),
- ▶ *Reconstruction* : Reconstruction de grilles (en volumique) ou de surfaces (en surfacique) à partir de données d'acquisition,
- ▶ *Modélisation* : Phase permettant d'assurer certaines propriétés sur l'objet reconstruit. (exemple : surface topologiquement correcte, i.e. sans trous, sans auto-intersections, ...).

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

Rappels

Traitements

Logiciel

Difficultés et limites

Fonctionnalités de base

- ▶ Lire des fichiers OBJ et PGM3D,
- ▶ Les visualiser,
- ▶ Permettre une navigation dans la scène 3D,
- ▶ Gérer la transparence des objets, pour voir leur intérieur,
- ▶ Avoir plusieurs modes de visualisation.

Décimation

- ▶ Réduire le nombre de sommets,
- ▶ **Objectif** : Compacter des données d'acquisition trop importantes.

Réparation de maillages

- ▶ Supprimer les triangles dégénérés,
- ▶ Ré-orienter les triangles orientés de façon incohérente,
- ▶ Réparer les trous,
- ▶ **Objectif** : Corriger la topologie d'un maillage pour le rendre *manifold*.

Point technique

- ▶ C++ / Qt,
- ▶ OpenGL,
- ▶ CGAL::Polyhedron [1].

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

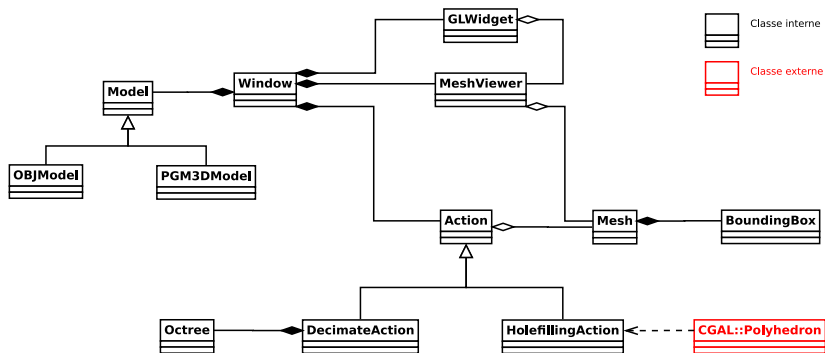
Rappels

Traitements

Logiciel

Difficultés et limites

Diagramme de classes



Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

Rappels

Traitements

Logiciel

Difficultés et limites

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

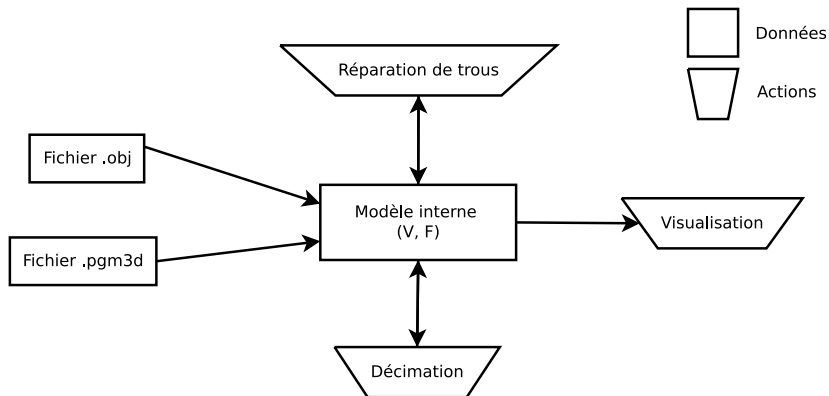
Rappels

Traitements

Logiciel

Difficultés et limites

Général



Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

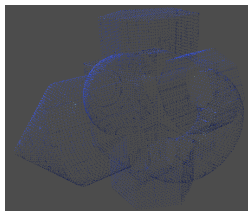
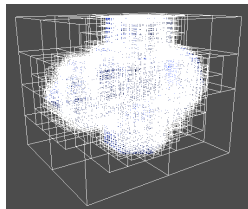
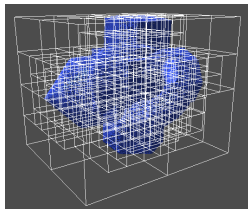
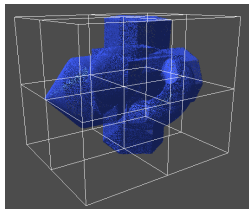
Rappels

Traitements

Logiciel

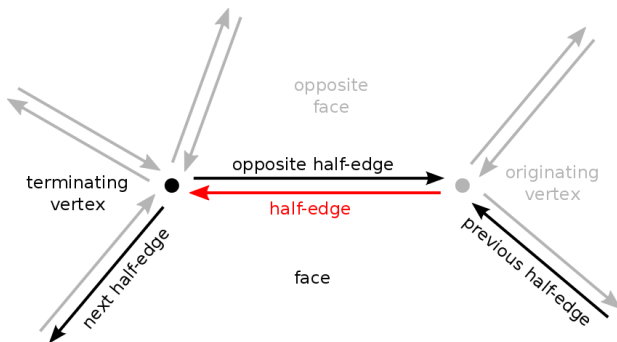
Difficultés et limites

Principe de décimation ¹ [3]



¹Source du modèle 3D : Bruno Dutailly

Structure en demi-arêtes [2]



Récapitulatif

	Décimation	Réparation de maillages
Structure	<i>Ocree</i> maison	CGAL::Polyhedron [1]
Avantage	Connaissance parfaite de l'objet	Bénéficier du savoir-faire de la bibliothèque CGAL
Inconvénient	Tout coder	Apprendre à utiliser la bibliothèque

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

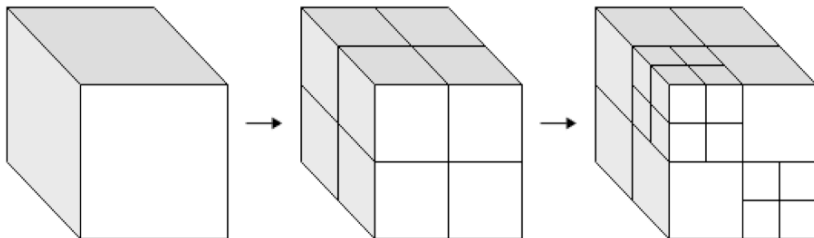
Rappels

Traitements

Logiciel

Difficultés et limites

Décimation - Principe



procedure BUILD(vertices)

Input: Un nuage de points.

Output: Résultat de la décimation.

 bbox = computeBBox(vertices)

 buildNode(vertices, bbox)

end procedure

▷ Calcule la bounding box des sommets.

Décimation - Construction d'un noeud

```
procedure BUILDNODE(vertices, bbox)
```

```
Input: Un ensemble de points à traiter et la bounding box du noeud courant.
```

```
Output: La décomposition récursive de la bounding box en 8 bounding box égales.
```

```
    if bbox.numVertices <= 10 then
```

```
        return
```

```
    end if
```

```
    bboxessub = divideBBox(vertices, bbox)
```

▷ Condition d'arrêt.

```
    for all bboxsub in bboxessub do
```

```
        if bboxsub.numVertices > 0 then
```

```
            buildNode(vertices, bboxsub)
```

```
        end if
```

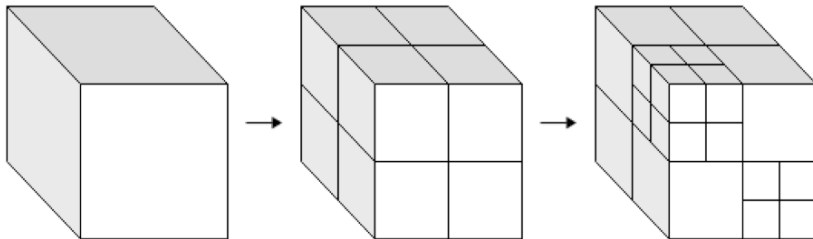
```
    end for
```

▷ Calcul des sous-bounding boxes
et de leur nombre de sommets.

▷ Etape de récursion.

```
end procedure
```

Décimation - Répartition - Approche naïve



```
function DIVIDEBBOX(vertices, bbox)
```

Input: Un ensemble de points à traiter et une *bounding box*.

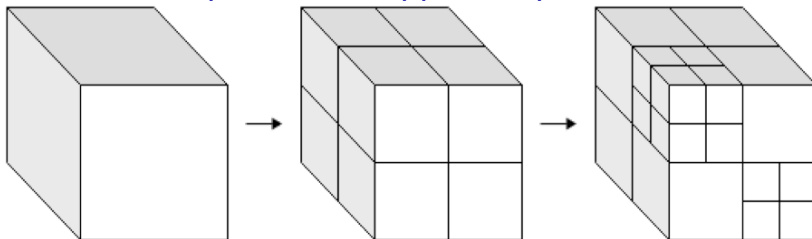
Output: La décomposition de la *bounding box* et le calcul du nombre de sommets par sous *bounding box*.

```
  for all  $bbox_{sub}$  in  $bbox.sub()$  do
    for all vertex in vertices do
      if belongs(vertex,  $bbox_{sub}$ ) then
         $bbox_{sub}.numVertices = bbox_{sub}.numVertices + 1$ 
      end if
    end for
  end for
  return  $bbox.sub()$ 
```

▷ Vérification chère en terme de calcul.

```
end function
```

Décimation - Répartition - Approche plus efficace



```
function BUILDNODE(vertices, bbox, start, end)
```

Input: Un ensemble de points à traiter et une *bounding box*, plus des bornes de recherche.

Output: La décomposition de la *bounding box* et le calcul du nombre de sommets par sous *bounding box*.

```
    bboxessub = bbox.sub()
```

```
    for all vertex in vertices[start, end] do
```

```
        bboxIdxsub = computeIndex(vertex, bbox)
```

```
        bboxes[bboxIdxsub].numVertices = bboxes[bboxIdxsub].numVertices + 1
```

```
    end for
```

```
    sortVertices(vertices[start, end], bboxessub)
```

```
    return bbox.sub()
```

```
end function
```

Réparation de trous - Principe

procedure FILLHOLES(*mesh*)

Input: Maillage troué à réparer.

Output: Maillage dont les trous ont été remplis.

HEStruct = *buildHE*(*mesh*)

holes = *detectHoles*(*HEStruct*)

for all *hole* in *holes* **do**

fillHole(*hole*)

end for

end procedure

Réparation de trous - Remplissage

```
procedure FILLHOLE(hole)
```

```
Input: Un trou à traiter dans le maillage.
```

```
Output: Le trou rempli.
```

```
  while notFilled(hole) do
```

```
    plane = makePlane(hole.vertex(0), hole.vertex(1), hole.vertex(2))
```

```
    vertex = notInPlane(hole, plane)
```

```
    if vertex == null then
```

```
      fillEntireHole(hole)
```

```
    else
```

```
      face = createFace(hole.vertex(0), vertex)
```

```
      addFace(hole, face)
```

```
    end if
```

```
    updateHole(hole)
```

```
  end while
```

```
end procedure
```

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

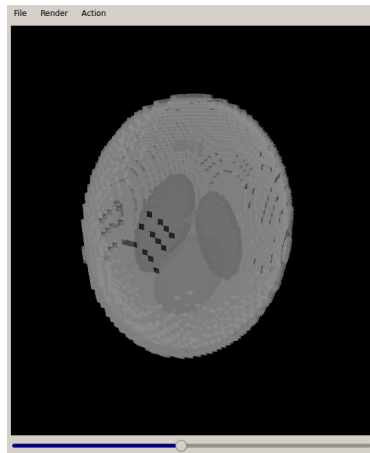
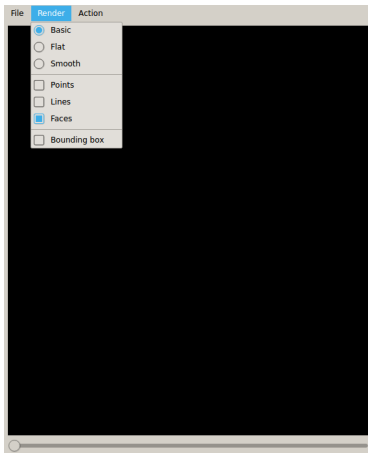
Rappels

Traitements

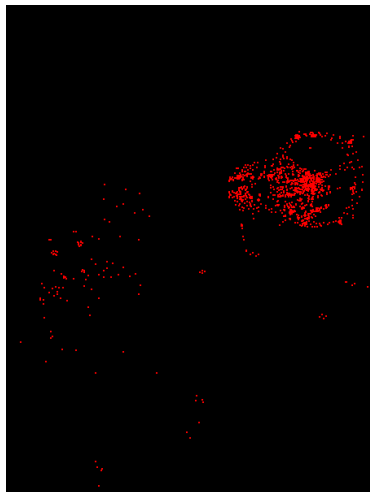
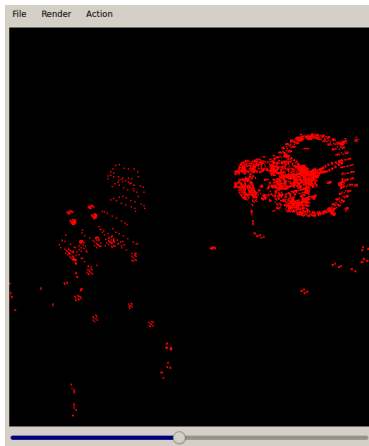
Logiciel

Difficultés et limites

Interface [4]

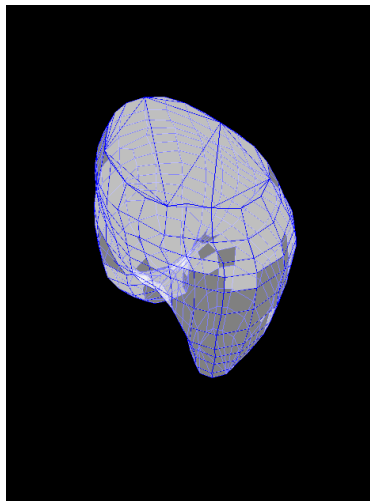
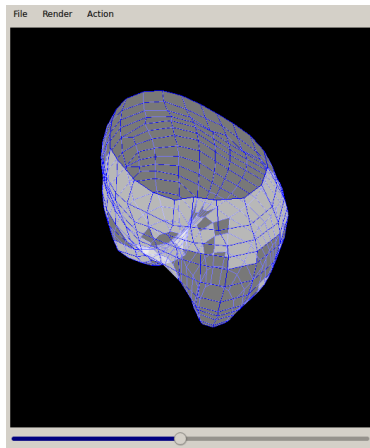


Décimation ²



²Source du modèle 3D : <http://people.sc.fsu.edu/~jburkardt/data/obj/obj.html>

Réparation de trous ³



³Source du modèle 3D : Fabzat

Plan

Introduction

Contexte

Cahier des charges

Architecture

Fonctionnement

Général

Rappels

Traitements

Logiciel

Difficultés et limites

Difficultés rencontrées

- ▶ Conception de l'architecture avec Qt,
- ▶ Apprentissage de `CGAL::Polyhedron` plus long que souhaité.

Limites

- ▶ *Pattern MVC* pas appliqué partout,
- ▶ Fuites de mémoire au niveau des contrôleurs d'action,
- ▶ Utilisation d'OpenGL 2.x,
- ▶ Seule la réparation de trous a été implémentée.

Et pour terminer...

Merci pour votre attention
Any questions ?

Références



CGAL::Polyhedron, 2016.

<http://doc.cgal.org/latest/Polyhedron/index.html>.



Halfedge principle, 2016.

<http://pointclouds.org/blog/nvcs/martin/>.



Pierre Bénard. Décimation par octree, 2016.

<http://www.labri.fr/perso/pbenard/>.



Lawrence A Shepp and Benjamin F Logan. The fourier reconstruction of a head section. *IEEE Transactions on Nuclear Science*, 21(3):21–43, 1974.